

LECTURE 2 • MOBILE APPLICATION AND DEVELOPMENT (0613-409)

Object-Oriented Programming in Java

Classes, objects, and the four pillars every Android Activity relies on

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science

Prerequisite for Activities

TODAY'S AGENDA

Building blocks of Java OOP

01

Classes & Objects

Blueprints, instances, fields, methods

02

Methods & Constructors

Overloading, initialization, a live demo

03

The Four Pillars

Abstraction, Encapsulation, Inheritance, Polymorphism

04

Bridge to Android

Where each pillar shows up in an Activity

WHY NOW?

Every Android Activity you'll write is already using OOP

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) { ... }  
}
```

class
MainActivity is a class

extends
Inheritance from AppCompatActivity

@Override
Polymorphism – replacing inherited behavior

PART 1 – CLASSES & OBJECTS

The Java Class

A class is a blueprint or prototype from which objects are created. It defines the state (fields) and behavior (methods) of a real-world thing.

```
["public"] ["abstract"|"final"] "class" Class_name
    ["extends" object_name] ["implements" interface_name]
"{
    // properties declarations
    // behavior declarations
}"
```

Fields = state (what an object knows) • Methods = behavior (what an object does)

ONE BLUEPRINT, MANY INSTANCES

The Java Object



An object stores its state in fields (variables) and exposes its behavior through methods. Hiding internal state is known as data encapsulation.

```
MainActivity a1 = new MainActivity();
```

WHAT'S INSIDE A CLASS

Class Members: Variables & Methods

Variable (field)

Member variables in a class — called fields.

- Class variable — static member
- Object variable — instance member
- Local variable — inside a method/block
- Parameter — inside a method declaration

Method

```
[ "public" | "private" | "protected" ]  
[ "static" | "abstract" ]  
return_type name(params) {  
    // actions  
}  
  
public int add2(int x) {x+=2; return x;}
```

PART 2 – METHODS & CONSTRUCTORS

Four types of Java methods

1 • Constructor

Creates objects with fields initialized to starting values

2 • Accessor

Reads a field's value — conventionally getField()

3 • Mutator

Sets a field's value — conventionally setField()

4 • Helper

Supports modular code structure in a unit of work

SAME NAME, DIFFERENT SIGNATURE

Method Overloading

Java distinguishes between methods by their signature: method name, argument type, and number of arguments.

```
public class TouchPointer {  
    public void draw(int i) { ... }  
    public void draw(double f) { ... }  
    public void draw(int i, int j) { ... }  
}
```

All three are named draw() — Java picks the right one at compile time based on the arguments you pass.

BUILDING AN OBJECT

Constructors

Called automatically with the "new" keyword to initialize a new object's fields. Similar to methods, with three key differences:

No return type

Not even void — the value is the object itself

No return statement

The body never returns a value explicitly

Name = class name

First line calls this() or super(), or the compiler inserts super()

No constructor written? Java auto-generates a parameterless default constructor.

LIVE DEMO

Lab: ConstructTest

```
public class ConstructTest {  
    int value1;  
    int value2;  
    ConstructTest() {  
        value1 = 10;  
        value2 = 20;  
    }  
    public void display() {  
        System.out.println("Value1 ≡ " + value1);  
        System.out.println("Value2 ≡ " + value2);  
    }  
    public static void main(String[] args) {  
        ConstructTest c1 = new ConstructTest();  
        c1.display();  
    }  
}
```

new ConstructTest()
creates the object

c1.display()
activates the method

Console output
Inside Constructor
Value1 ≡ 10
Value2 ≡ 20

PART 3

The Four Pillars of OOP

Object-oriented programming first appeared at MIT in the late 1950s; Smalltalk (Xerox PARC, 1970s) coined the term. It models how real-world objects are built from smaller objects.

Abstraction

Encapsulation

Inheritance

Polymorphism

PILLAR 1

Abstraction

Manages complexity by hiding implementation. You don't need to know how every part of a computer works to use it — abstraction focuses on the outside view of an object (the interface).

```
interface Customer {  
    void printCustomer(int i);  
}
```

In Android: you call `startActivity()` without knowing how the OS actually switches screens.

PILLAR 2

Encapsulation

Binds state and behavior together and keeps both safe from outside interference and misuse — hiding a field so only the class's own methods can touch it.

```
class Customer {  
    private String name;  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

private hides the field · getters/setters control how it's read and changed

PILLAR 3

Inheritance

One class acquires the properties of another, supporting hierarchical classification. Java uses "extends" to inherit properties and behavior from a superclass into a subclass.

```
class Customer {  
    String name;  
    String getName() {...}  
}
```



```
class MemberCustomer  
    extends Customer {  
    int collectedPoints;  
}
```

In Android: class MainActivity extends AppCompatActivity — you inherit an entire lifecycle for free.

PILLAR 4

Polymorphism

The ability to group object behavior from different types. One object can be treated like another, even from a different hierarchy — the exact type is decided at runtime.

```
interface IFace_Printable { void printNow(); }

class CustomerInfo implements IFace_Printable {
    @Override
    public void printNow() { ... }
}
```

In Android: `@Override public void onCreate(...)` — you replace the default lifecycle behavior with your own.

LIVE DEMO

Lab: one interface, two behaviors

```
class CustomerInfo implements
    IFace_Printable {
    public void printNow() {
        System.out.println("Read Customer Table");
    }
}

class ProductInfo implements
    IFace_Printable {
    public void printNow() {
        System.out.println("Reading Product Table");
    }
}
```

```
class PrintServer {
    private IFace_Printable p;
    PrintServer(IFace_Printable p) {
        this.p = p;
    }
    public void printNowWrapper() {
        p.printNow();
    }
}

new PrintServer(customerInfo)
    .printNowWrapper();
new PrintServer(productInfo)
    .printNowWrapper();
```

PrintServer never checks "is this a CustomerInfo or a ProductInfo?" — it just calls printNow() and the right code runs.

MOBILE ARCHITECTURE & YOUR FIRST PROJECT

Two toolkits, one goal: reading real Android code

So far

The Android stack — Kernel → HAL → Runtime →
Framework → Apps

Also so far

Java OOP — classes, objects,
inheritance, polymorphism

Today

Put both together and build a real Android project

WHERE YOUR JAVA CODE ACTUALLY RUNS

The Android Runtime (ART)

Your Java/Kotlin classes compile to DEX bytecode, not standard JVM bytecode. Each app runs in its own ART instance — its own process, its own memory space.

AOT compilation

Ahead-of-time — compiled once at install

JIT compilation

Just-in-time — optimizes hot code as it runs

Dalvik (retired)

ART's predecessor, before Android 5.0

Why this matters: it's the same "runtime" box from the software stack we covered earlier — now you know what's actually happening inside it.

TWO KINDS OF BUILDING BLOCKS

Native Libraries vs. Java API Framework

Native C/C++ Libraries

The layer below your code — you use it, never write it directly

SQLite · OpenGL ES · WebKit · Media Framework · Libc

Java API Framework

The layer you build with directly, every day

View System · Resource Manager · Activity Manager · Content Providers

This is abstraction in action: you call the framework's methods without knowing the native code underneath.

WHAT ANDROID STUDIO GENERATES FOR YOU

Anatomy of an Android project

manifests/	AndroidManifest.xml
java/	MainActivity.java / .kt
res/layout/	activity_main.xml
res/values/	strings.xml, colors.xml
res/mipmap/	launcher icons
Gradle Scripts/	build.gradle

Code and resources live apart on purpose — a UI string can change without touching a single line of Java.

LIVE DEMO

Creating a new Android project

- 1** New Project → Empty Views Activity
- 2** Name the project & set the package name (e.g. com.diu.helloandroid)
- 3** Choose the save location and language (Java or Kotlin)
- 4** Set the minimum SDK — this decides how many devices can install your app

TWO NUMBERS THAT MATTER

SDK & project configuration

minSdkVersion

The oldest Android version your app will install on

targetSdkVersion

The version you've tested against and optimize for

```
<uses-sdk android:minSdkVersion="24" android:targetSdkVersion="34" />
```

The launcher icon (res/mipmap) and app label are set here too — Android Studio's project wizard configures all of this for you.

NOW THE OOP PAYS OFF

Creating your first Activity

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

extends AppCompatActivity
Inheritance — you get a full lifecycle for free

@Override onCreate()
Polymorphism — you replace default startup behavior

setContentView()
Abstraction — connects code to the XML layout underneath

FROM CODE TO SCREEN

Running & stopping the app

Run ►

Gradle builds the APK, installs it on the AVD, launches MainActivity

Stop ■

Ends the process from Android Studio — cleaner than force-stopping on device

Logcat

Where `println()` and crash traces show up while it runs

Every run re-triggers `onCreate()` — this is where you'll watch the Activity lifecycle in action later.

LIVE DEMO

Modifying the example app

```
res/values/strings.xml  
<string name="hello">  
    Hello, Android!  
</string>
```

Change it, then Run ► again

```
<string name="hello">  
    Hello, [Your Name]!  
</string>
```

No Java changes needed — this is why strings, colors and dimensions live in resource files instead of hardcoded in code.

CODE AND CONTENT, KEPT APART

Reviewing layout & resource files

res/layout/

XML files describing screens — activity_main.xml

res/values/strings.xml

All user-facing text, in one place — translation-ready

res/values/colors.xml

Named color constants reused across layouts

res/mipmap/

Launcher icons, at multiple resolutions

A layout XML references a string by name (@string/hello), never the literal text — so the same layout works in any language.

WHERE THIS LEAVES US

You can now read and run a real Android project

✓ **Project structure & the ART runtime**

✓ **Creating, running & stopping an app**

✓ **Editing resources safely**

Next: Software Development Platform — screen sizes, resource qualifiers, and building your first real UI.

BEFORE NEXT CLASS

What to do this week

Rebuild it yourself

Create your own new project from scratch,
end to end, without following along

Edit three resources

Change a string, a color, and the app name —
then re-run

Review both OOP demos

ConstructTest and the polymorphism demo
— you'll need them again soon

Next lecture: Software Development Platform — directory structure, resource qualifiers & screen sizes

Questions?

Java OOP · Mobile Architecture & Your First Android Project

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science